SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Mukundhan V.R *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

mukundhan

Question 1: Hospital Shift Assignment (Backtracking Search)

1.1 Problem formulation as a CSP

- **Variables:** D1, D2, D3 (one per doctor).
- **Domain of each variable:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N.
- **Constraints:**
  - C1: D1 ≠ Night
  - C2: D2 must work before D3, i.e. shift(D2) < shift(D3)
  - C3: Only one doctor per shift (All-Different)
  - C4: D3 ≠ Morning
  - C5: Every doctor gets exactly one shift (complete assignment, 3 doctors – 3 shifts)

Node consistency (unary constraints applied first): C1 reduces D1 to {M, A}; C4 reduces D3 to {A, N}. D2 keeps {M, A, N}. C2, C3 and C5 are binary / global constraints that are checked during search.

Search settings: variable order D1 → D2 → D3, value order M → A → N, depth-first search. After every assignment, all constraints whose variables are fully assigned are checked; if one fails the algorithm backtracks to the previous choice.

1.2 Backtracking trace with constraint checks

Step	Partial assignment tried	Constraint checks	Result
1	D1 = M	C1: M ≠ N ✓. No other variable assigned yet.	Accept → go deeper
2	D1=M, D2 = M	C3: Morning already used by D1 ✗	Reject, try next value
3	D1=M, D2 = A	C3 ✓ (A is free). C2 cannot be checked yet (D3 unassigned).	Accept → go deeper
4	D1=M, D2=A, D3 = M	C4: D3 ≠ Morning ✗ (also C3 ✗)	Reject
5	D1=M, D2=A, D3 = A	C3: Afternoon already used by D2 ✗	Reject
6	D1=M, D2=A, D3 = N	C1 ✓, C3 ✓ (M, A, N all different), C4 ✓ (N ≠ M), C2 ✓ (A < N), C5 ✓ (all assigned)	GOAL – valid schedule found

Continuing the search (to confirm all solutions / show backtracking):

Step	Assignment tried	Constraint checks	Result
7	Backtrack to D2; D1=M, D2 = N	C3 ✓ (N free)	Accept
8	D1=M, D2=N, D3 = M	C4 ✗	Reject
9	D1=M, D2=N, D3 = A	C3 ✓, C4 ✓, but C2: N < A is false ✗	Reject → D3 exhausted → backtrack to D1
10	D1 = A	C1: A ≠ N ✓	Accept
11	D1=A, D2 = M	C3 ✓	Accept
12	D1=A, D2=M, D3 = M	C3 ✗ and C4 ✗	Reject
13	D1=A, D2=M, D3 = A	C3 ✗ (Afternoon used by D1)	Reject
14	D1=A, D2=M, D3 = N	C1 ✓, C2 ✓ (M < N), C3 ✓, C4 ✓, C5 ✓	Second valid solution

Step	Assignment tried	Constraint checks	Result
15	D1=A, D2 = A / N …	D2 = A ✗ (C3). D2 = N: D3 = M ✗ (C4), D3 = A ✗ (C3 & C2) → dead end	Backtrack
16	D1 = N	C1 ✗ (D1 cannot work Night)	Pruned (already removed by node consistency)

1.3 Final valid schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Verification: C1 ✓ (D1 is on Morning, not Night); C2 ✓ (D2 Afternoon < D3 Night); C3 ✓ (one doctor per shift); C4 ✓ (D3 is on Night, not Morning); C5 ✓ (every doctor has exactly one shift).

Note: The constraints actually admit two schedules – (D1=Morning, D2=Afternoon, D3=Night) and (D1=Afternoon, D2=Morning, D3=Night). Backtracking with the stated variable/value ordering reaches the first one first, so it is reported as the answer. In both, D3 works Night.

Possible improvement: Using Forward Checking / MRV would remove D3's values M and A early and finish in fewer steps, but plain backtracking is sufficient for this small problem.

Question 2: Robot in a 5×5 Grid (BFS and Manhattan-Heuristic Search)

2.1 Problem definition and assumed grid

The question paper does not print the grid, so the following 5×5 grid is assumed. Cells are written as (row, column) with (0,0) at the top-left. The same procedure applies to any other obstacle layout.

S	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	■ X	(1,2)	■ X	(1,4)
(2,0)	■ X	(2,2)	(2,3)	(2,4)
(3,0)	(3,1)	(3,2)	■ X	(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	**G**

- **Start S** = (0,0); **Goal G** = (4,4); **Obstacles (X)** = (1,1), (1,3), (2,1), (3,3).
- **Actions:** Up, Down, Left, Right (expanded in this order). **Step cost** = 1. Obstacles and cells outside the grid cannot be entered.
- **Heuristic:** Manhattan distance $h(n) = |r - 4| + |c - 4|$. It is admissible and consistent for 4-directional moves with unit cost, so $h(S) = 8$ is a lower bound on the optimal cost.
- **State space:** the $25 - 4 = 21$ free cells; goal test: $n = (4,4)$.

Note on the question: plain BFS uses a FIFO queue and does not use a heuristic; a priority queue with the Manhattan heuristic belongs to Greedy / A* search. Both are therefore shown: 2.2 BFS (FIFO queue) and 2.3 A* (priority queue ordered by $f = g + h$).

2.2 BFS – step-by-step node expansion (FIFO queue)

A node is marked visited when it is generated, so every cell enters the queue only once. Format: cell [depth].

#	Node expanded	Children generated	Queue after step (front → back)
1	(0,0) [0]	(1,0), (0,1)	(1,0), (0,1)
2	(1,0) [1]	(2,0) — (1,1) is an obstacle	(0,1), (2,0)
3	(0,1) [1]	(0,2) — (1,1) obstacle	(2,0), (0,2)
4	(2,0) [2]	(3,0) — (2,1) obstacle	(0,2), (3,0)
5	(0,2) [2]	(1,2), (0,3)	(3,0), (1,2), (0,3)
6	(3,0) [3]	(4,0), (3,1)	(1,2), (0,3), (4,0), (3,1)
7	(1,2) [3]	(2,2) — (1,1), (1,3) obstacles	(0,3), (4,0), (3,1), (2,2)
8	(0,3) [3]	(0,4) — (1,3) obstacle	(4,0), (3,1), (2,2), (0,4)
9	(4,0) [4]	(4,1)	(3,1), (2,2), (0,4), (4,1)
10	(3,1) [4]	(3,2) — (2,1) obstacle; (4,1) already visited	(2,2), (0,4), (4,1), (3,2)
11	(2,2) [4]	(2,3) — (3,2) already visited	(0,4), (4,1), (3,2), (2,3)
12	(0,4) [4]	(1,4)	(4,1), (3,2), (2,3), (1,4)
13	(4,1) [5]	(4,2)	(3,2), (2,3), (1,4), (4,2)
14	(3,2) [5]	none new — (3,3) obstacle	(2,3), (1,4), (4,2)

#	Node expanded	Children generated	Queue after step (front → back)
15	(2,3) [5]	(2,4) — (1,3), (3,3) obstacles	(1,4), (4,2), (2,4)
16	(1,4) [5]	none new — (2,4) already visited	(4,2), (2,4)
17	(4,2) [6]	(4,3)	(2,4), (4,3)
18	(2,4) [6]	(3,4)	(4,3), (3,4)
19	(4,3) [7]	(4,4) = GOAL generated [8]	(3,4), (4,4) → goal found

BFS result: 19 nodes expanded. Following parent pointers back from G:

(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Path length = 8 moves, total cost = 8 × 1 = 8. BFS is optimal here because every step has the same cost, and it explores level by level so the first time G is reached is via a shortest path.

2.3 A* with Manhattan heuristic – priority queue updates

Priority = f(n) = g(n) + h(n). Ties are broken by the smaller h (closer to goal), then by insertion order. Format: cell(g+h=f).

#	Node popped (lowest f)	Successors generated (g+h=f)	Priority queue after step
1	(0,0) f=8	(1,0) g1+h7=8, (0,1) g1+h7=8	(1,0)=8, (0,1)=8
2	(1,0) f=8	(2,0) 2+6=8 [(1,1) obstacle]	(2,0)=8, (0,1)=8
3	(2,0) f=8	(3,0) 3+5=8 [(2,1) obstacle]	(3,0)=8, (0,1)=8
4	(3,0) f=8	(4,0) 4+4=8, (3,1) 4+4=8	(4,0)=8, (3,1)=8, (0,1)=8
5	(4,0) f=8	(4,1) 5+3=8	(4,1)=8, (3,1)=8, (0,1)=8
6	(4,1) f=8	(4,2) 6+2=8 [(3,1) already open with better g=4]	(4,2)=8, (3,1)=8, (0,1)=8
7	(4,2) f=8	(4,3) 7+1=8, (3,2) 7+3=10	(4,3)=8, (3,1)=8, (0,1)=8, (3,2)=10
8	(4,3) f=8	(4,4) 8+0=8 [(3,3) obstacle]	(4,4)=8, (3,1)=8, (0,1)=8, (3,2)=10
9	(4,4) f=8	GOAL popped from the priority queue	Stop

A* expands only 9 nodes (against 19 for BFS) and returns the same path and cost, because the Manhattan heuristic keeps f = 8 along the whole solution path – the heuristic is exact here.

2.4 Optimal path and cost

S	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	■ X	(1,2)	■ X	(1,4)
(2,0)	■ X	(2,2)	(2,3)	(2,4)
(3,0)	(3,1)	(3,2)	■ X	(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	**G**

Cells shaded blue form the path found.

Item	Value
Optimal path	(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)
Moves	Down, Down, Down, Down, Right, Right, Right, Right
Optimal cost	8 (equals the Manhattan lower bound, so no cheaper path exists)
Alternative equal-cost path	(0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4), also cost 8

Property	BFS	A* (Manhattan)
Frontier	FIFO queue	Priority queue on f = g + h
Nodes expanded	19	9
Path cost	8 (optimal)	8 (optimal)
Complete / Optimal	Yes / Yes (unit costs)	Yes / Yes (admissible, consistent h)

Question 3: Autonomous Rescue Robot in a Disaster Building

a) Analysis of the constraints and their effect on decision making

Constraint	Effect on the robot's decisions
Partially observable environment; sensing only adjacent cells	The robot cannot compute a full plan in advance. It must keep an internal map (belief state) of what it has seen, treat unseen cells as unknown, and decide using incomplete information. This makes the problem an online search / exploration problem.
Moves: Up, Down, Left, Right	Defines the action set (branching factor ≤ 4) and a discrete 4-connected state space. The Manhattan distance to G is a usable, admissible heuristic.
Blocked cells (obstacles)	Remove actions from the state space. Newly sensed obstacles invalidate the current plan, so the robot must be able to re-plan and sometimes backtrack from dead ends.
Move cost = 1; entering a risky zone costs 1 + 2 = 3	Path cost is no longer the number of steps, so BFS is unsuitable. The robot compares cost, not length: a risky cell costs as much as three normal steps, so it is used only when a detour costs more.
Minimise total cost with safe navigation	Gives an optimisation objective (least-cost path) plus a safety preference. Safety is built into the cost function: risky cells are penalised heavily, and cells known to be blocked are never entered.
Dynamic knowledge update (online search)	Planning and acting are interleaved: sense → update map → plan → act one step → repeat. The agent needs memory (map, costs) and a policy for unknown cells.

Priority of the constraints (most to least binding): (1) obstacles / safety – hard constraints that can never be violated; (2) partial observability – shapes the type of algorithm; (3) cost minimisation with the risk penalty – decides between feasible paths; (4) move set – defines the neighbourhood. Hard constraints are filtered first, then the cheapest feasible option is chosen.

b) Solution approach: Online Uniform-Cost / A* Search Agent

Chosen strategy: an online search agent that repeatedly runs Uniform Cost Search (or A* with the Manhattan heuristic, which is UCS guided toward G) on its current known map, executes only the first step, senses, and re-plans. This is similar in spirit to D* Lite / online A*.

Step cost function:

Cell entered	Cost
Normal free cell	1
Risky zone	1 + 2 = 3
Blocked cell (known obstacle)	∞ (not allowed)
Unknown cell	Assumed free (cost 1), corrected when sensed – "free-space assumption"

Steps of the algorithm:

1. **Initialise:** current cell = S, known map = {S}, visited set empty, total cost = 0.
2. **Sense:** read the four adjacent cells and classify each as free, blocked or risky; add them to the known map.
3. **Update knowledge:** if a cell on the current plan is now known to be blocked, discard the plan.
4. **Plan:** run UCS / A* from the current cell to G over the known map using g(n) = accumulated cost (1 per move, +2 for a risky cell), with f(n) = g(n) + h(n), h = Manhattan distance to G (admissible because every step costs at least 1).

5. **Act:** execute only the first move of the cheapest path and add its cost to the total.
6. **Dead ends:** if no path exists in the known map, move to the nearest frontier (unexplored) cell and explore; if the robot is boxed in, backtrack to the last cell that had an unexplored neighbour (as in LRTA* / online DFS).
7. **Repeat** Sense → Update → Plan → Act until the current cell is G (survivor reached). Return the path travelled and the total cost.

Pseudocode:

```
function ONLINE_RESCUE_AGENT(S, G):
  pos <- S ; known <- {S} ; cost <- 0
  while pos != G:
    percept <- SENSE(neighbours(pos))       # free / blocked / risky
    UPDATE(known, percept)
    plan <- A_STAR(known, pos, G,           # UCS + Manhattan heuristic
                step_cost = 1 (+2 if risky), blocked = INF)
    if plan is empty:
      plan <- PATH_TO_NEAREST_FRONTIER(known, pos)
      if plan is empty: return FAILURE      # survivor unreachable
    next <- plan[0]
    pos  <- MOVE(next) ; cost <- cost + step_cost(next)
  return SUCCESS, cost
```

Small illustration: if the cell directly toward G turns out to be risky (cost 3) and a safe route needs two extra moves (extra cost 2), the robot compares total costs after sensing and picks the cheaper, i.e. it takes the safe route unless the safe route costs more than the risk penalty. If a corridor is sensed to be blocked, the robot marks it, re-plans and moves via the next cheapest known route.

c) Application of AI concepts

(i) Intelligent agent

An agent perceives its environment through sensors and acts on it through actuators. The rescue robot is a **goal-based and utility-based agent with a model (map) of the world**: it maintains internal state, has a goal (reach survivor at G) and a utility measure (lowest cost, avoid risk).

PEAS element	Rescue robot
Performance measure	Reach G (survivor found); minimise total path cost (moves + risk penalties); avoid blocked / dangerous cells; minimise time
Environment	Grid of rooms in a damaged building with obstacles and risky zones
Actuators	Wheels / motors for Up, Down, Left, Right movement; rescue mechanism
Sensors	Proximity / obstacle sensors, camera or thermal sensor covering the 4 adjacent cells; position (odometry)

Percept-action cycle: percept (adjacent cells) → update internal map → decide the best next move by search → action (move one cell) → new percept.

(ii) Rationality

A rational agent selects, for every percept sequence, the action expected to maximise its performance measure given its built-in knowledge. The robot is rational because:

- it uses all percepts collected so far (the updated map) and never repeats an invalid action such as walking into a known obstacle;
- it picks the move on the cheapest known path to G, trading off distance against the risk penalty, so it maximises the performance measure with the information available;

- it explores unknown cells only when necessary (information gathering), and it learns from new percepts by re-planning.

Rationality here is **bounded**: the robot cannot be optimal over the whole unseen building, but it is optimal with respect to its current knowledge and cost model, and it never takes an action it knows to be worse.

(iii) Environment type

Property	Classification	Reason
Observability	Partially observable	Sees only adjacent cells
Agents	Single agent	Only one robot; survivors and debris are not decision-making agents
Determinism	Deterministic (assumed)	A move leads to the intended neighbour; it becomes stochastic if wheels slip or debris shifts
Episodic / sequential	Sequential	Each move changes the position and affects all future costs and choices
Static / dynamic	Static (assumed) / semi-dynamic	Layout is assumed fixed while planning, but aftershocks can change it; hence re-planning
Discrete / continuous	Discrete	Grid cells, four fixed actions
Known / unknown	Partly unknown	The map is not known in advance and is learned while moving

d) Justification of the chosen approach

- **Handles partial observability:** an online agent that interleaves sensing, planning and acting does not need the full map in advance, which an offline planner would require.
- **Handles non-uniform costs:** UCS / A* minimise the sum of step costs including the +2 risk penalty. BFS only minimises the number of steps and may route through risky cells.
- **Optimal and complete on known information:** UCS / A* with an admissible heuristic (Manhattan distance, as every step costs ≥ 1) guarantee the cheapest path on the known map, and re-planning ensures the robot finishes if a safe path exists in a finite grid.
- **Efficient:** the Manhattan heuristic focuses the search toward the goal, which expands far fewer nodes than plain UCS (as seen in Question 2: 9 vs 19 nodes).
- **Adaptive and safe:** blocked cells get infinite cost and risky cells a high cost, so safety is part of the decision; new obstacles trigger re-planning rather than failure.

Why not the alternatives?

Strategy	Limitation for this problem
BFS	Ignores step costs and risk; optimal only when all costs are equal
DFS / Depth-limited	Not optimal, may wander through risky zones and waste cost
Greedy best-first	Ignores cost already incurred; not optimal and can enter risky cells
Offline A* / UCS alone	Needs the complete map beforehand, which a partially observable environment does not provide
Online agent with UCS / A* (chosen)	Handles unknown map, risk costs, and re-planning; optimal with respect to current knowledge

Conclusion: an online search agent that plans with Uniform Cost Search (accelerated with the admissible Manhattan heuristic as A*) and updates its map after each sensing step is the most suitable strategy. It respects all hard constraints, minimises total cost including the risk penalty, copes with partial observability and is rational with respect to the information available.